Web Applications I 2025/26 – Exam #1 (deadline 2026-06-22 at 23:59)

“Last Race”

⚠️ PRELIMINARY VERSION – Leave any questions as COMMENTS and do not delete or resolve any existing comments. The final version will be published on 2026-06-05.

Design and implement a web application for a single-player game inspired by the board game “Race the Rails”.

In the game, the player is assigned a starting station and a destination station, which vary in each game, within a fictional underground network. The player must plan and execute a valid route before time runs out, gaining or losing coins along the way due to random events. The goal is to reach the destination with the highest possible score.

The game is based on an **underground network** consisting of a fixed set of stations connected by metro lines. Both stations and lines have unique names. A simplified example of the network could be:

- Red Line: Centrale <-> Porta Velaria <-> Crocevia del Falco <-> Piazza delle Lanterne
- Blue Line: Centrale <-> Fontana Oscura <-> Borgo Sereno <-> Viale dei Mosaici
- Green Line: Porta Velaria <-> Fontana Oscura <-> Torre Cinerea <-> Campo dell'Eco
- Yellow Line: Piazza delle Lanterne <-> Torre Cinerea <-> Viale dei Mosaici <-> Campo dell'Eco

The names of the stations and lines, as well as the connections between stations, are left to the student. The underground network must have *at least 4 lines*, *at least 12 stations*, and *at least 3 interchange stations*, that is, stations served by more than one line. The network does not change during the game.

On a probabilistic basis, at least 8 different events may occur during a segment; each event consists of a description and an effect, a positive or negative integer from -4 to +4. Students may define the events as they wish. Examples of events may include: “Quiet journey, 0 coins”, “Wrong platform, -2 coins”, “Kind passenger, +1 coin”.

The network and the events must be stored on the server and retrieved by the client at the appropriate times.

The application allows users to play multiple games. Each game starts with 20 coins and consists of the following phases:

1. **Setup.** The player sees the network map with all stations, their connections, and the lines. When the player is ready to play, they move on to the next phase.
2. **Planning.** The player sees three elements on the page:
    a. the network map, showing only the stations with their names but **without** the lines connecting them;
    b. a starting station and a destination station, randomly assigned by the server, where the destination station must be reachable in the network from the starting station with a minimum distance of at least 3 ~~stops~~ segments between them (as an example,

Centrale -> Porta Velaria -> Crocevia del Falco -> Piazza delle Lanterne counts as 3 segments, involving 4 stops);

c. the list of all **segments**, that is, pairs of connected stations, for example, Porta Velaria—Fontana Oscura.

From the beginning of this phase, the player has 90 seconds to scroll through the list of pairs, mentally reconstruct the network, and build their route by selecting the segments in sequence. Each segment may be selected only once. The **route** must start from the assigned starting station and end at the assigned destination station.

Before the 90 seconds expire, the player must submit the route they have built. If time runs out, the planning phase automatically ends with the route built up to that point, even if it is incomplete or invalid.

A route is **valid** when it starts and ends at the assigned stations and each segment is reachable through one of the lines, with line changes possible only at interchange stations. Routes are valid if they involve the same station more than once, but they must not involve any segment more than once.

3. **Execution.** The web application validates the submitted route and, for each segment of the journey, that is, each step from one station to the next, randomly selects one event from those available and applies its effect to the player's total number of coins. The web application shows the steps one at a time, in sequence, displaying the unexpected event that occurred and the updated coin total.

   If the submitted route is invalid or incomplete, this phase is skipped and the player loses all 20 coins in their possession, thus obtaining a score of zero coins.

4. **Result.** The player is shown the final score, which corresponds to the coins remaining in the game. If the final score is negative, it will be stored and shown as zero. The player can choose whether to start a new game.

Registered users, that is, users who have logged in, can play as many games as they want. In addition, the **best** result from their games must be displayed in a general ranking on a dedicated page of the application.

Anonymous users, that is, site visitors, can only view the game instructions, without showing the network map. They will not have access to any functionality available to registered users.

The organization of these specifications into different screens, and possibly different routes, is left to the student.

Project requirements

- The application architecture and source code must be developed by adopting the best practices in software development, in particular, those relevant to single-page applications

(SPA) using React and HTTP APIs. APIs should be carefully protected and the front-end should not receive unnecessary information.

- The application should be designed for a desktop browser. Responsiveness for mobile devices is not required nor evaluated.
- The project must be implemented as a React 19 application that interacts with an HTTP API implemented in Node+Express. The Node version must be the one used during the course (24.x, LTS). The database must be stored in a SQLite file. The programming language must be JavaScript.
- The communication between client and server must follow the "two servers" pattern, by properly configuring CORS, and React must run in "development" mode with Strict Mode activated.
- The evaluation of the project will be carried out by navigating the application. Neither the behavior of the "refresh" button, nor the manual entering of a URL (except /) will be tested, and their behavior is undefined. Also, the application should never "reload" itself as a consequence of normal user operations.
- The root directory of the project must contain a README.md file and have two subdirectories (client and server). The project must be started by running the two commands: "`cd server; nodemon index.js`" and "`cd client; npm run dev`". A template for the project directories is already available in the exam repository. You may assume that nodemon is globally installed. No other global modules will be available.
- The whole project must be submitted on GitHub, on the same repository created by GitHub Classroom.
- The project **must not include** the `node_modules` directories. They will be re-created by running the "`npm install`" command right after "`git clone`".
- The project may use popular and commonly adopted libraries (for example, day.js, react-bootstrap, etc.), if applicable and useful. All required libraries must be correctly declared in the package.json file, so that the `npm install` command might install them.
- User authentication (login and logout) and API access must be implemented with `Passport.js` and session cookies. The credentials should be stored in an encrypted and salted form. The user registration procedure is not requested nor evaluated.

Quality requirements

In addition to the implementation of the required application functionality, the following quality requirements will be evaluated:

- Database design and organization.
- Design of the HTTP APIs.
- Organization of React components and routes.
- Correct usage of React patterns (functional behavior, hooks, state, context, and effects). Avoiding direct manipulation of the DOM is included in these rules.
- Code clarity.
- Absence of errors (and warnings) in the client console (except those caused by errors in the imported libraries).
- Absence of application crashes or unhandled exceptions.
- Essential data validation (in Express and React).

- Basic usability and user-friendliness.
- Commit history in the GitHub repository
- Originality of the solution.

NOTE: In the evaluation of the project, during the oral discussion, it is expected that the project has been fully developed by the candidate student, who should therefore possess detailed knowledge about the code, and especially about the design decisions and the adopted functions or solutions. It is not forbidden to collaborate with other students (but not sharing whole parts of the project), nor to utilize AI coding assistance, but the student still has 100% responsibility for the knowledge, understanding and capacity to explain the code.

Database requirements

The project database must be designed by the student and must be pre-populated (seeded) with the following initial data:

- at least 4 lines
- at least 12 stations
- at least 3 interchange stations
- at least 8 different events
- at least 3 registered users
- 2 of the registered users must have already successfully played some games.

Contents of the README.md file

The README.md file must contain the following information (a template is available in the project repository). Generally, each information item should take no more than 2-3 lines.

1. Server-side:
    a. A list of the HTTP APIs offered by the server, with a short description of the parameters and the exchanged objects.
    b. A list of the database tables, with their purpose.
2. Client-side:
    a. A list of 'routes' for the React application, with a short description of the purpose of each route.
    b. A list of the main React components.
3. Overall:
    a. Two screenshots of the application, one **with the general ranking page** and one **during a game**. The screenshot must be embedded in the README by linking the image committed in the project repository.
    b. Usernames and passwords of the registered users.
    c. The possible usage of AI during the development of the project, describing for what purpose AI has been used and how their output has been verified and/or adapted. If you didn't use any AI, please state it.

Submission procedure

To correctly submit the project, you must:

- **Be enrolled** in the exam call.
- Use the provided **link** to **join the classroom** on GitHub Classroom (i.e., correctly **associate** your GitHub username with your student ID) and **accept the assignment**. (https://classroom.github.com/a/ekpOfkrq)
- **Push the project** into the **main branch** of the repository created for you by GitHub Classroom. The last commit (the one you wish to be evaluated) must be **tagged** with the tag **`final`** (note: `final` is all-lowercase with no spaces, and it is a git 'tag', nor a 'commit message').

Note: to tag a commit, you may use (from the terminal) the following commands:

```
# ensure the latest version is committed
git commit -m "...comment..."
git push

# add the 'final' tag and push it
git tag final
git push origin --tags
```

Alternatively, you may insert the tag from GitHub's web interface (follow the link 'Create a new release').

To test your submission, these are the exact commands that the teachers will use to download and run the project. You may wish to test them on a clean directory:

```
git clone ...yourCloneURL...
cd ...yourProjectDir...
git pull origin main  # just in case the default branch is not main
git checkout -b evaluation final # check out the version tagged with
'final' and create a new branch 'evaluation'
(cd client ; npm install; npm run dev)
(cd server ; npm install; nodemon index.js)
```

Ensure that all the needed packages are downloaded by the npm install commands. Be careful: if some packages are installed globally on your computer, they might not be listed as dependencies. Always check it in a clean installation.

Be aware that Linux is case-sensitive for file names, while Windows and macOS are not. Double-check the case of import statements and all file names.